

Django REST Framework

Complete Practical Notes — Setup to API Versioning

Prepared by Abhi Shah

16 Topics • Step-by-Step Instructions • Code • Line-by-Line Syntax • Sample Output

How to Use This Document

Each topic follows the same structure: Concept (what & why), Step-by-Step (what to do, in order), Code (exact code to write, file by file), Syntax Explained (the meaning of every new keyword/line), and Sample Output (what you should see once it works).

“Sample Output” boxes are shown in a dark terminal/browser style so they are easy to spot while revising. They illustrate the shape of the response — your own field values (names, ages, tokens) will differ.

Contents

Topic 1: Introduction to Django REST Framework.....	
Topic 2: Project Setup: Installing DRF & Creating the App.....	
Topic 3: Creating a Model.....	
Topic 4: Registering the Model in Django Admin.....	
Topic 5: Your First API View — @api_view and Response.....	
Topic 6: Serializing Model Data with serializers.Serializer.....	
Topic 7: Creating Records — the POST API.....	
Topic 8: Updating & Deleting — PUT, PATCH, DELETE.....	
Topic 9: ModelSerializer — Less Code, Same Result.....	
Topic 10: Routers & ViewSets — One Class for Full CRUD.....	
Topic 11: Pagination — Limiting Results Per Page.....	
Topic 12: JWT Authentication — Setup.....	
Topic 13: Using the JWT Token in Postman & Token Lifetime.....	
Topic 14: Custom Permissions — IsAdminOrReadOnly.....	
Topic 15: Per-Action Permissions with get_permissions().....	
Topic 16: API Versioning — Running v1 and v2 Side by Side.....	

Topic 1: Introduction to Django REST Framework

Concept

Django REST Framework (DRF) is a toolkit built on top of Django for building Web APIs. A normal Django view returns an HTML page; a DRF view returns data (usually JSON) that any client can consume — a React app, a mobile app, or a tool like Postman. DRF adds four main building blocks on top of plain Django:

- Serializers – convert Python/model objects to JSON and validate incoming JSON back into Python data.
- Views – function-based (`@api_view`) or class-based (`ViewSet`) request handlers.
- Routers – automatically generate URL patterns for a `ViewSet`'s CRUD actions.
- Authentication & Permissions – decide who is making the request and what they're allowed to do.

Notes

- Keep this order in mind while reading the rest of this document: Model → Serializer → View → URL. Every feature we add later plugs into one of these four layers.

Topic 2: Project Setup: Installing DRF & Creating the App

Concept

Every DRF project starts the same way: install the package, create a Django project, then create at least one app that will hold your models, views, and URLs.

Step-by-Step

1. Install Django REST Framework into your Python environment.
2. Create a new Django project called core1.
3. Register 'rest_framework' inside INSTALLED_APPS in settings.py.
4. Create a new app called home.
5. Register 'home' inside INSTALLED_APPS as well.
6. Wire the home app's view into the project's urls.py.

Code

Terminal

```
pip install djangorestframework
python -m django startproject core1
python manage.py startapp home
```

core1/settings.py

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'home',
]
```

core1/urls.py

```
from home import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Syntax Explained — What Each Line Means

- **pip install djangorestframework** — Downloads and installs the DRF package so 'rest_framework' becomes importable in your code.

- `python -m django startproject core1` — Generates a new Django project named `core1` with the default `settings.py`, `urls.py` and `wsgi.py` files.
- `python manage.py startapp home` — Generates a new app folder called `'home'` containing its own `models.py`, `views.py` and `admin.py`.
- `INSTALLED_APPS` — A list in `settings.py` telling Django which apps are active. Adding `'rest_framework'` here turns on DRF's features project-wide.
- `from home import views` — Imports the `views.py` module of the `home` app so its functions can be attached to a URL.
- `path('', views.home, name='home')` — Maps the root URL (`''`) to the `home` function. `name='home'` lets other code refer to this URL by name instead of typing it out.

Topic 3: Creating a Model

Concept

A model is a Python class that Django turns into a database table. Each class attribute becomes a column.

Step-by-Step

1. Open `home/models.py`.
2. Define the Student model with its fields.
3. Generate and apply the migration so the table is actually created in the database.

Code

`home/models.py`

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    surname = models.CharField(max_length=200)
```

Terminal

```
python manage.py makemigrations
python manage.py migrate
```

Syntax Explained — What Each Line Means

- **`models.Model`** — The base class every model must inherit from; it is what gives the class database powers such as `.save()` and `.objects.all()`.
- **`CharField(max_length=100)`** — A short text field. `max_length` is required and sets the maximum number of characters the database column will store.
- **`IntegerField()`** — Stores whole numbers only.
- **`makemigrations`** — Scans your models for changes and writes a migration file — a blueprint describing how the database should change.
- **`migrate`** — Applies pending migration files to the actual database, creating or altering tables to match your models.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Terminal output

```
Migrations for 'home':
  home/migrations/0001_initial.py
    - Create model Student

Operations to perform:
  Apply all migrations: admin, auth, contenttypes, home, sessions
Running migrations:
  Applying home.0001_initial... OK
```


Topic 4: Registering the Model in Django Admin

Concept

Django ships with a ready-made admin website that can list, add, edit and delete rows for any model you register — no HTML needed.

Step-by-Step

1. Import the Student model into admin.py.
2. Register it with `admin.site.register()`.
3. Create a superuser account.
4. Run the server and log in at /admin.

Code

home/admin.py

```
from .models import Student
admin.site.register(Student)
```

Terminal

```
python manage.py createsuperuser
python manage.py runserver
```

Syntax Explained — What Each Line Means

- **admin.site.register(Student)** — Tells the built-in admin site to expose this model, automatically generating its list, add, edit and delete pages.
- **createsuperuser** — An interactive command that asks for a username, email and password, then creates an admin login you can use at /admin.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browser — <http://127.0.0.1:8000/admin/>

Django administration

HOME

Students

+ Add

Change

AUTHENTICATION AND AUTHORIZATION

Groups	+ Add	 Change
Users	+ Add	 Change

Topic 5: Your First API View — @api_view and Response

Concept

A DRF view is a normal Python function turned into an API endpoint using the `@api_view` decorator. Instead of returning HTML, it returns a `Response` object, which DRF renders as JSON (or as a nicely formatted page when opened in a browser — the “Browsable API”).

Code

home/views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view()
def home(request):
    return Response({'status': 200, 'message': 'Hello from django'})
```

Syntax Explained — What Each Line Means

- **@api_view()** — Converts a plain function into a DRF view. With empty parentheses it defaults to allowing GET requests only. Without this decorator, `Response()` would not work correctly.
- **Response(data)** — DRF's replacement for Django's `HttpResponse`. It automatically serializes the dict into JSON and, when viewed in a browser, renders a readable HTML page around it.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browser — <http://127.0.0.1:8000/>

```
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json

{
  "status": 200,
  "message": "Hello from django"
}
```

Topic 6: Serializing Model Data with `serializers.Serializer`

Concept

A serializer converts complex data (querysets, model instances) into simple Python data types (dicts/lists) that can be rendered into JSON, and does the reverse when receiving data. `serializers.Serializer` is the manual, “write every field yourself” style — it mirrors Django's `forms.Form`.

Code

home/serializers.py

```
from rest_framework import serializers

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField()
    age = serializers.IntegerField()
    surname = serializers.CharField()
```

home/views.py

```
@api_view(['GET'])
def home(request):
    students = Student.objects.all()
    student_serializer = StudentSerializer(students, many=True)
    return Response(student_serializer.data)
```

Syntax Explained — What Each Line Means

- **`serializers.Serializer`** — The base class for a manually-defined serializer, where every field is declared explicitly.
- **`serializers.CharField()` / `IntegerField()`** — Declares the expected type of each field on the serializer — separate from (but usually matching) the model's own field types.
- **`Student.objects.all()`** — An ORM query that fetches every row of the `Student` table as a `QuerySet`.
- **`StudentSerializer(students, many=True)`** — `many=True` tells the serializer it has received a list/queryset of objects rather than one single object, so it outputs a JSON array.
- **`.data`** — The converted, plain Python data (a list of dicts here) that `Response()` can safely turn into JSON.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browser — `http://127.0.0.1:8000/`

```
[
```

```
{ "name": "Aditi", "age": 21, "surname": "Shah"},  
{ "name": "Rahul", "age": 22, "surname": "Mehta"}  
]
```

Topic 7: Creating Records — the POST API

Concept

To accept new data, a view must read the incoming request body, validate it with a serializer, and save it. Since `serializers.Serializer` does not know how to save data by itself, you must write `create()` and `update()` methods yourself.

Code

core1/urls.py

```
path('create/', views.student_create, name='post_student'),
```

home/views.py

```
@api_view(['POST'])
def student_create(request):
    serializer = StudentSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
    return Response(serializer.errors)
```

home/serializers.py (add methods)

```
def create(self, validated_data):
    return Student.objects.create(**validated_data)

def update(self, instance, validated_data):
    instance.name = validated_data.get('name', instance.name)
    instance.age = validated_data.get('age', instance.age)
    instance.surname = validated_data.get('surname', instance.surname)
    instance.save()
    return instance
```

Syntax Explained — What Each Line Means

- **request.data** — DRF's parsed version of the incoming request body. Unlike Django's `request.POST`, it works for JSON, form-data and any other content type.
- **StudentSerializer(data=request.data)** — Passing `data=` (instead of an instance) puts the serializer into 'incoming data' mode — for validating and saving, not for reading existing objects.
- **serializer.is_valid()** — Runs every field's validation (required, type checks, etc.) and returns True/False. Must be called before `.save()` or trusting `.data` on write.

- **`serializer.save()`** — Calls `create()` (new record) or `update()` (existing instance) automatically, depending on whether the serializer was given an instance.
- **`serializer.errors`** — A dictionary of validation error messages, available when `is_valid()` returns `False`.
- **`**validated_data`** — Unpacks the validated dictionary into keyword arguments, e.g. `Student.objects.create(name=..., age=..., surname=...)`.
- **`validated_data.get('name', instance.name)`** — Uses the new value if it was supplied, otherwise falls back to the existing value — this is what makes partial updates safe.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Postman — POST /create/

Body (raw JSON):

```
{ "name": "Kiran", "age": 23, "surname": "Joshi" }
```

Response 200 OK:

```
{ "name": "Kiran", "age": 23, "surname": "Joshi" }
```

Topic 8: Updating & Deleting — PUT, PATCH, DELETE

Concept

PUT replaces a full record, PATCH updates only the fields you send, and DELETE removes the row entirely. Each needs its own URL and view when working with a plain serializer.

Code

core1/urls.py

```
path('<int:pk>/put/', views.student_update, name='put_data'),
path('<int:pk>/patch/', views.student_partial_update, name='patch_data'),
path('<int:pk>/delete/', views.student_delete, name='delete_data'),
```

home/views.py

```
@api_view(['PUT'])
def student_update(request, pk):
    student = Student.objects.get(pk=pk)
    serializer = StudentSerializer(student, data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
    return Response(serializer.errors)

@api_view(['PATCH'])
def student_partial_update(request, pk):
    student = Student.objects.get(pk=pk)
    serializer = StudentSerializer(student, data=request.data, partial=True)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
    return Response(serializer.errors)

@api_view(['DELETE'])
def student_delete(request, pk):
    student = Student.objects.get(pk=pk)
    student.delete()
    return Response({'message': 'deleted'})
```

Syntax Explained — What Each Line Means

- **<int:pk>** — A URL path converter: captures an integer from the URL and passes it into the view function as the pk argument.

- **`Student.objects.get(pk=pk)`** — Fetches exactly one record by primary key. Raises an error if no matching row exists — in production this is usually wrapped in try/except or replaced with `get_object_or_404`.
- **`StudentSerializer(student, data=request.data)`** — Passing both an instance AND data puts the serializer into 'update' mode: it will update that specific object instead of creating a new one.
- **`partial=True`** — Tells the serializer that not all fields are required in this request — only the ones provided will be validated and changed.
- **`student.delete()`** — Permanently removes that row from the database.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browser links to test manually

```
http://127.0.0.1:8000/2/put/  
http://127.0.0.1:8000/2/patch/  
http://127.0.0.1:8000/2/delete/
```

Topic 9: ModelSerializer — Less Code, Same Result

Concept

ModelSerializer inspects a model and automatically generates its fields plus create()/update() logic, removing almost everything you wrote by hand in Topics 6 and 7.

Code

```
home/serializers.py

from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'
```

Syntax Explained — What Each Line Means

- **serializers.ModelSerializer** — A serializer subclass that reads a Django model and automatically builds matching serializer fields plus working create()/update() methods.
- **class Meta:** — A configuration-only inner class that Django/DRF looks for by convention. It is not used like a normal class — it just holds settings such as model and fields.
- **model = Student** — Tells the serializer which model to base its auto-generated fields on.
- **fields = '__all__'** — Includes every field from the model. You could instead pass a list like ['name', 'age'] to expose only specific fields.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Behaviour

Same JSON output as Topics 6 & 7, e.g.:
{
 "id": 1,
 "name": "Aditi",
 "age": 21,
 "surname": "Shah"
}

...but create()/update() no longer need to be written manually.

Topic 10: Routers & ViewSets — One Class for Full CRUD

Concept

A `ViewSet` groups `list/retrieve/create/update/partial_update/destroy` logic into a single class instead of five separate functions. A `Router` then automatically generates all the matching URLs for that class.

Code

home/views.py

```
from rest_framework import viewsets

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

core1/urls.py

```
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('students', views.StudentViewSet, basename='student')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include(router.urls)),
]
```

Syntax Explained — What Each Line Means

- **`viewsets.ModelViewSet`** — Bundles the full CRUD action set (`list`, `retrieve`, `create`, `update`, `partial_update`, `destroy`) into one class.
- **`queryset`** — The base set of rows this `ViewSet` operates on.
- **`serializer_class`** — Which serializer to use for converting data in and out of this `ViewSet`.
- **`DefaultRouter()`** — Automatically builds the standard `list/detail` REST URL patterns and adds a browsable API root page.
- **`router.register('students', StudentViewSet, basename='student')`** — Registers the URL prefix `'students'` to be handled entirely by `StudentViewSet`. `basename` names the generated URL patterns internally.
- **`include(router.urls)`** — Plugs all of the router's auto-generated URL patterns into the project's main `urlpatterns` list.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browsable API — <http://127.0.0.1:8000/students/>

GET /students/ → list of all students + an HTML form to POST a new one
GET /students/1/ → single student + PUT, PATCH and DELETE buttons
(all generated automatically — no extra URLs or view functions needed)

Topic 11: Pagination — Limiting Results Per Page

Concept

When a table has many rows, returning all of them in one response is slow and wasteful. Pagination splits results across pages that the client requests one at a time.

Code

home/pagination.py

```
from rest_framework.pagination import PageNumberPagination

class StudentPagination(PageNumberPagination):
    page_size = 2
    page_size_query_param = 'size'
    max_page_size = 4
```

home/views.py

```
from .pagination import StudentPagination

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    pagination_class = StudentPagination
```

Syntax Explained — What Each Line Means

- **PageNumberPagination** — Built-in DRF class that paginates results using simple ?page=2 style URLs.
- **page_size** — The default number of items returned per page.
- **page_size_query_param** — Lets the client override the page size via a query parameter, e.g. ?size=5.
- **max_page_size** — A hard upper limit on how large a page a client can request via page_size_query_param, preventing abuse such as ?size=10000.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Browser — <http://127.0.0.1:8000/students/?page=2>

```
{
  "count": 6,
  "next": "http://127.0.0.1:8000/students/?page=3",
  "previous": "http://127.0.0.1:8000/students/?page=1",
  "results": [
```

```
{
  "id": 3, "name": "Kiran", "age": 23, "surname": "Joshi"},
  {"id": 4, "name": "Neha", "age": 20, "surname": "Verma"}
]
```

Topic 12: JWT Authentication — Setup

Concept

JSON Web Tokens (JWT) let a client prove who it is on every request by attaching a signed token, without the server needing to store session data. `django-rest-framework-simplejwt` adds this to DRF.

Step-by-Step

1. Install the `simplejwt` package and run migrate.
2. Set `JWTAuthentication` as the `DEFAULT_AUTHENTICATION_CLASSES` in `settings.py`.
3. Add `token-obtain` and `token-refresh` URLs.
4. Create a user (via admin) and request a token using their username/password.

Code

Terminal

```
pip install django-rest-framework-simplejwt
python manage.py migrate
```

core1/settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

core1/urls.py

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
```

Syntax Explained — What Each Line Means

- **`django-rest-framework-simplejwt`** — A third-party package that implements JWT-based authentication for DRF.
- **`DEFAULT_AUTHENTICATION_CLASSES`** — Tells DRF which method(s) to use to identify 'who is making this request' — here, by reading a JWT from the Authorization header.
- **`TokenObtainPairView`** — A ready-made view that accepts a username and password and returns an access token + refresh token pair.

- **TokenRefreshView** — Accepts a valid refresh token and issues a brand-new access token once the old one has expired.
- **.as_view()** — Converts a class-based view into a callable function that Django's URL dispatcher can use.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Postman — POST /api/token/

Body: { "username": "abhi", "password": "*****" }

Response 200 OK:

```
{  
  "refresh": "eyJhbGciOiJIUzI1NiIs...",  
  "access": "eyJhbGciOiJIUzI1NiIs..."  
}
```


Topic 13: Using the JWT Token in Postman & Token Lifetime

Concept

Once you have an access token, it must be attached to every protected request. You can also configure how long access and refresh tokens stay valid.

Step-by-Step

1. In Postman, open the Authorization tab of your request.
2. Choose type 'Bearer Token'.
3. Paste the access token value from Topic 12.
4. Send the request — it will now succeed instead of returning 401.

Code

```
core1/settings.py

from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
}
```

Syntax Explained — What Each Line Means

- **timedelta(minutes=30)** — A Python object representing a duration. Here it configures how long an access token remains valid before being rejected as expired.
- **ACCESS_TOKEN_LIFETIME / REFRESH_TOKEN_LIFETIME** — SIMPLE_JWT settings controlling the expiry windows for the short-lived access token and the longer-lived refresh token.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

```
Postman

Without a token:
401 Unauthorized
{ "detail": "Authentication credentials were not provided." }

With Authorization: Bearer <access token>:
200 OK — requested data is returned normally
```

Topic 14: Custom Permissions — IsAdminOrReadOnly

Concept

A permission class decides, for every incoming request, whether the current user is allowed to proceed. Custom permissions let you mix rules — for example, anyone logged in can read, but only staff can write.

Code

```
home/permissions.py

from rest_framework.permissions import BasePermission, IsAuthenticated, IsAdminUser

class IsAdminOrReadOnly(BasePermission):
    def has_permission(self, request, view):
        if not (request.user and request.user.is_authenticated):
            return False

        SAFE_METHODS = ("GET", "HEAD", "OPTIONS")
        if request.method in SAFE_METHODS:
            return True

        return bool(request.user and request.user.is_staff)
```

Syntax Explained — What Each Line Means

- **BasePermission** — The parent class every custom permission must inherit from.
- **has_permission(self, request, view)** — DRF automatically calls this for every incoming request. Returning True allows the request through; returning False results in a 403 Forbidden.
- **request.user** — The currently authenticated user object, or an AnonymousUser if nobody is logged in.
- **request.user.is_authenticated** — Boolean — True only if a valid, logged-in user made this request.
- **SAFE_METHODS** — A conventional tuple of HTTP methods considered 'read-only' because they don't change any data.
- **request.user.is_staff** — Django's built-in flag marking an admin/staff account.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Behaviour

```
Logged-in, non-staff user → GET succeeds, POST/PUT/DELETE → 403 Forbidden
Logged-in, staff user    → GET, POST, PUT, DELETE all succeed
Not logged in            → every request → 403 Forbidden
```

Topic 15: Per-Action Permissions with `get_permissions()`

Concept

Sometimes different actions on the same `ViewSet` need different permission rules — e.g. reading is open to any logged-in user, but writing is admin-only. Overriding `get_permissions()` lets one `ViewSet` apply different rules per action.

Code

home/views.py

```
from .permissions import IsAuthenticated, IsAdminUser

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    def get_permissions(self):
        if self.action in ("list", "retrieve"):
            permission_classes = [IsAuthenticated]
        else:
            permission_classes = [IsAdminUser]
        return [permission() for permission in permission_classes]
```

Syntax Explained — What Each Line Means

- **`self.action`** — A string DRF automatically sets on a `ViewSet` ('list', 'retrieve', 'create', 'update', 'partial_update', 'destroy') telling you which operation is currently running.
- **`get_permissions(self)`** — Overriding this method lets you return a different set of permission instances depending on `self.action`, instead of one fixed `permission_classes` list for the whole `ViewSet`.
- **`[permission() for permission in permission_classes]`** — DRF expects permission instances, not classes, so each class in the list is called with `()` to instantiate it before being returned.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Behaviour

```
Non-staff user → GET /students/          → 200 OK
Non-staff user → POST /students/         → 403 Forbidden
                {"detail": "You do not have permission to perform this action."}
```

Topic 16: API Versioning — Running v1 and v2 Side by Side

Concept

Versioning lets you improve your API (v2) without breaking clients still relying on the old behaviour (v1). Here this is done manually by giving each version its own router and URL prefix.

Code

core1/urls.py

```
router_v1 = DefaultRouter()
router_v1.register('students', views.StudentViewSet, basename='student-v1')

router_v2 = DefaultRouter()
router_v2.register('students', views.StudentViewSetV2, basename='student-v2')

urlpatterns += [
    path('v1/', include(router_v1.urls)),
    path('v2/', include(router_v2.urls)),
]
```

home/views.py

```
from .serializers import StudentSerializer, StudentSerializerV2

class StudentViewSetV2(StudentViewSet):
    serializer_class = StudentSerializerV2
```

home/serializers.py

```
class StudentSerializerV2(serializers.ModelSerializer):
    full_name = serializers.SerializerMethodField()

    class Meta:
        model = Student
        fields = ['id', 'name', 'surname', 'age', 'full_name']

    def get_full_name(self, obj):
        return f"{obj.name} {obj.surname}"
```

Syntax Explained — What Each Line Means

- **`path('v1/', include(router_v1.urls))`** — Each API version lives under its own URL prefix (v1/, v2/), so old clients calling /v1/... keep working unchanged even after /v2/... is introduced.

- `class StudentViewSetV2(StudentViewSet)` — Inherits everything from the v1 ViewSet (queryset, permissions) and overrides only `serializer_class`, avoiding duplicated code.
- `SerializerMethodField()` — A read-only field whose value comes from a method you define (`get_<field_name>`) rather than directly from a model attribute — used here for a computed value.
- `get_full_name(self, obj)` — DRF automatically calls this for the `full_name` field; `obj` is the specific model instance currently being serialized.
- `f"{obj.name} {obj.surname}"` — A Python f-string — builds a string by inserting variable values directly inside the curly braces.

Sample Output

Illustrative output — run the steps above on your machine to see your own data.

Comparing versions

```
GET /v1/students/1/
{"id": 1, "name": "Aditi", "surname": "Shah", "age": 21}

GET /v2/students/1/
{"id": 1, "name": "Aditi", "surname": "Shah", "age": 21, "full_name": "Aditi Shah"}
```